SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

Team: 199 XiaoBai

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

Our proposed system, Z.Clear, functions as an AI-powered compliance engine that focuses on automating the extraction, standardization, and management of critical data from unstructured documents, such as logistics emails and commercial invoices, which are commonly characterized by inconsistent formats and varied content structures.

1.1. Purpose:

The system analysis document highlights the technical scope and design-related decision behind the Z.Clear development.

The key element of the system that has been covered are as followed:

1) **Architecture**
    a) The system is designed as a modular, AI-enabled document intelligence platform built on a microservices architecture to support scalable and resilient trade operations. The core components include:
        i) **Unstructured Document Parsing Engine** for extracting and structuring key data from logistics emails, commercial invoices, and other irregular document formats
        ii) **Task Management and Status Tracking Service** for monitoring processing stages and ensuring full workflow transparency
        iii) **Multilingual Interface Layer (API/UI)** for seamless user interaction and localization across Chinese, English, and Bahasa Melayu
        iv) **High Availability Parsing Layer** comprising a primary AI-driven extraction engine and an independent backup parsing mechanism to ensure operational continuity
        v) **AI Reasoning and Extraction Layer (GLM-based)**
        Powered by a General Language Model from Z.ai, this layer performs deep semantic parsing, enabling the system to interpret unstructured text, recognize shorthand and industry jargon, and extract context-aware trade data fields. The GLM serves as the primary reasoning engine for data extraction and normalization.
    b) This architecture enables scalability, fault tolerance, and real-time responsiveness, while allowing seamless integration with external enterprise systems and trade-related platforms.
2) **Data Flows**
    a) Users upload documents or input shipment-related text through a web-based or mobile interface. The data is transmitted to the backend API, where authentication and request validation are performed before routing it to the document parsing engine.
    b) The parsing engine processes the unstructured content using AI models to extract and standardize key fields such as Goods Description, HS Code, and Weight. The structured data is then stored in a persistent database (e.g., MariaDB) for further use and audit purposes.

© UMHackathon 2026

Email: umhackathon@um.edu.my

c) Throughout the process, the task status tracking service updates the workflow stages in real time—Received, Extracting, Auditing, Completed, or Error—and communicates these updates back to the user interface.
d) In cases where the primary AI model is unavailable due to network restrictions or external dependencies, the system automatically switches to the backup parsing mechanism to continue processing without interruption.
e) Once processing is successfully completed, the system stores the finalized structured data and provides a completion status to the user. If errors occur, feedback is returned through the API for user review and corrective action.

3) **Model Process**
   a) The model represents the end-to-end workflow of the system's intelligent document processing capabilities. It begins with document ingestion, where unstructured inputs such as emails or invoices are submitted by the user.
   b) The AI-driven parsing engine then performs data extraction and normalization, transforming inconsistent and fragmented information into structured outputs. The system tracks progress continuously and ensures visibility across all stages of processing.
   c) If the primary AI model encounters limitations, the system dynamically activates the backup parsing mechanism to maintain continuity. The workflow concludes with validated, structured data that is ready for downstream usage, along with a system-generated completion status.
4) **Role of Reference**
   a) This document serves as a technical reference for developers, system architects, and testers to understand the system's overall design, operational workflows, and integration points. It provides a high-level blueprint of component interactions, data flow mechanisms, and the role of AI-driven and fallback parsing strategies.
   b) By outlining these elements, the document ensures consistent implementation, reliable system behavior, and effective validation across all stages of the platform's lifecycle.

1.2. Background:

SMEs in the logistics sector often rely on manual processes to extract customs-related data from unstructured sources such as emails, invoices, and chat messages. These inputs are typically inconsistent and contain shorthand or industry jargon, making data entry slow and error-prone. As trade volume increases, there is a need for an automated solution to convert unstructured text into structured, standardized data for efficient customs processing.

1. Previous Version:

The previous system depended on manual data entry, with limited support from basic rule-based tools such as keyword matching or regular expressions. These methods struggled with inconsistent formats,

© UMHackathon 2026

Email: umhackathon@um.edu.my

multilingual content, and context-dependent information. As a result, the workflow was inefficient, difficult to scale, and prone to human error.

2. *Changes in Major Architectural Components*

The new system introduces a modular, AI-enabled architecture that significantly enhances automation, scalability, and reliability. Key architectural changes include:

a. AI Reasoning Layer (GLM-Based): Integration of a General Language Model from Z.ai for intelligent parsing and contextual understanding.
b. AI-Driven Parsing Engine: Replaces rule-based methods with flexible unstructured data processing.
c. Task Tracking Service: Enables real-time monitoring of processing stages.
d. High Availability Layer: Adds a backup parsing mechanism for reliability.
e. API & Multilingual Layer: Supports system integration and multi-language interaction.

These changes enable automation, scalability, and continuous compliance monitoring across the entire export workflow.

3. *New capabilities that are only introduced in this*

The enhanced system delivers several new capabilities aligned with the described functionalities:

a. Context-aware data extraction from unstructured text
b. One-click retrieval of key customs fields (e.g., HS Code, Weight)
c. Automated data cleaning and standardization
d. Recognition of shorthand and industry jargon
e. Real-time task status tracking
f. Fault-tolerant processing with backup support

These capabilities significantly improve efficiency, reduce manual effort, and enhance the reliability of export compliance for SMEs.

1.3. *Target Stakeholder*

Stakeholders	***Roles***	***Expectations***

© UMHackathon 2026

Email: umhackathon@um.edu.my

SMEs / Exporters	Oversee operations and decision-making	Improved efficiency, reduced operational costs, and faster customs processing
Operations Staff (Data Entry / Documentation)	Input and verify shipment and customs data	Reduced manual workload, higher accuracy, and easier data extraction
Customs Brokers / Compliance Officers	Ensure correctness of customs declarations	Accurate, standardized data and compliance with regulations
IT / System Administrators	Maintain system infrastructure and integration	Stable system performance, easy maintenance, and secure integration with existing platforms
End Users (Logistics Personnel)	Upload documents and retrieve structured data	Simple interface, fast processing, and minimal errors

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	AI-Driven Export Compliance and Trade Facilitation Platform (Web and mobile-based application for SMEs to automate extraction and processing of customs data from unstructured inputs)
Architecture	Cloud-native modular architecture with API-first design, integrated with AI-driven processing services powered by a GLM from Z.ai for document parsing, data extraction, and compliance validation

The system is structured as a cloud-native, API-driven platform with two main client interfaces: a web application and a mobile application for SME users. Both clients communicate with a centralized backend API layer that handles authentication, request routing, and system integration.

The backend consists of modular services, including a document parsing service, task management and status tracking service, and data storage service. The core intelligence is powered by a GLM from Z.ai, which performs unstructured text parsing, data extraction, and normalization.

To ensure reliability, a high-availability parsing layer is implemented with a primary AI engine and a fallback parsing mechanism. The system is deployed on a cloud infrastructure, with structured data stored in a relational database (e.g., MariaDB), enabling scalability, fault tolerance, and seamless integration with external logistics systems.

2.1.2 LLM as Service Layer

The system integrates a GLM-based LLM service from Z.ai as a dedicated service layer within the backend architecture. This layer is exposed through internal APIs and acts as the core intelligence engine for all unstructured text understanding tasks.

The LLM service layer is responsible for semantic parsing of free-form inputs (emails, invoices, chat logs), extraction of key customs fields (e.g., Goods Description, HS Code, Weight), and normalization of shorthand and industry-specific jargon into structured formats.

© UMHackathon 2026

Email: umhackathon@um.edu.my

It is decoupled from application logic and accessed by downstream microservices such as the document processing service and data structuring service. This separation ensures scalability, reusability, and independent deployment of the model layer.

A fallback parsing mechanism is integrated alongside the LLM service to ensure continuity in cases of model latency or unavailability.

The following Dependency diagram includes:

2.1.3 Dependency Diagram

a. *How the prompts are being constructed and sent to the GLM API*

Prompts are built in the backend **Prompt Construction Layer** inside the Document Processing Service.
Each prompt includes:

- Raw unstructured text (email, invoice, chat)
- Extraction schema (e.g., Goods Description, HS Code, Weight)
- System instructions (format rules, normalization rules)
- Optional metadata (document type, origin, user context)

The final structured prompt is sent via internal API to the GLM service from Z.ai.

b. *What goes into the context window?*

The GLM context window contains:

- Chunked document text (if large input)
- Extraction schema definition (JSON structure)
- Domain rules (customs formatting rules)
- Relevant session/context data (if multi-document processing)
- Few-shot examples (optional for better accuracy)

c. *How is your system receiving, parsing the responses and passing to the next system component?*

- GLM returns structured JSON output
- Backend parses response in Document Processing Service
- Data Structuring Service validates and normalizes fields
- Task Service updates processing status (e.g., Extracted, Completed)
- Final structured data is stored in MariaDB (AWS RDS)

d. *Where the limitation of token are enforced or input are chunked before it actually reaches to GLM*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Token control is handled in a **pre-GLM middleware layer**:

- Estimates token usage before API call
- Splits input into semantic chunks if limit is exceeded
- Uses overlap strategy to preserve context
- Aggregates multiple GLM responses into one final structured output

e. *Outline of the major API calls between the system components*

- Frontend → API Gateway: `POST /documents/upload`
- API Gateway → Document Service: `POST /process-document`
- Document Service → GLM Service: `POST /glm/extract`
- Document Service → Database (MariaDB): `INSERT / UPDATE structured trade data`
- Task Service → Frontend: `GET /task-status`

f. *Do show how frontend, backend, services, database and external API interacts with each other*

- Frontend (Web/Mobile) sends requests via API Gateway
- Backend microservices process and coordinate workflow
- GLM service from Z.ai handles semantic extraction
- MariaDB stores structured customs data
- Object storage stores raw documents and logs
- Services communicate via REST APIs and internal service calls

g. Distributed deployment components

- The system is distributed across:
- Frontend (web/mobile clients)
- API Gateway (central routing layer)
- Microservices (document, task, data services)
- GLM inference service (isolated AI layer)
- MariaDB database (managed cloud RDS)
- Object storage (for raw documents)
- Each component is independently scalable and deployable.

2.2.2. Sequence Diagram

a. *walk through one user interaction flow or customer journey with the stakeholders*

- SME user uploads an unstructured document (email / invoice / chat text) via web or mobile app
- Request goes through **API Gateway**
- API Gateway forwards it to **Document Processing Service**

© UMHackathon 2026

Email: umhackathon@um.edu.my

- System creates a task ID and stores raw input
- **Task Service** marks status as *RECEIVED*
- Document Processing Service builds prompt (text + schema + rules)
- Input is checked for size; if large, it is chunked before processing
- Prompt is sent to GLM service from Z.ai
- GLM returns structured extraction output (JSON)
- Backend validates and parses response
- Data Structuring Service normalizes fields
- Final structured data is saved into **MariaDB**
- Task Service updates status to *COMPLETED*
- Frontend retrieves result and displays structured customs data to user

b. *Show the interactions step by step.*

- Frontend → API Gateway: submit document
- API Gateway → Document Service: route request
- Document Service → Task Service: create tracking task
- Document Service → GLM Service: send prompt
- GLM Service → Document Service: return structured output
- Document Service → Data Service: normalize & prepare data
- Data Service → MariaDB: persist structured record
- Task Service → Frontend: update status in real time

c. *Do demonstration of each key system features does operate from start to finish*

- **Unstructured Input Handling:** raw email/invoice is accepted directly
- **AI Extraction (GLM):** extracts HS Code, weight, description automatically
- **Chunking Mechanism:** large documents are split before sending to GLM
- **Data Cleaning:** values like "Wt: 80kg" converted to numeric format
- **Real-Time Tracking:** system shows RECEIVED → PROCESSING → COMPLETED
- **Structured Storage:** final output saved in MariaDB for retrieval

2.2. Technological Stack

2.2.1. **Frontend:** Streamlit / Next.js – Web-based user interface for SME users to upload unstructured documents, trigger extraction workflows, and view structured customs data and processing status in real time

2.2.2. **Backend:** FastAPI – RESTful API framework for handling microservices communication, document processing workflows, integration with the GLM service, and management of extraction and validation pipelines

© UMHackathon 2026

Email: umhackathon@um.edu.my

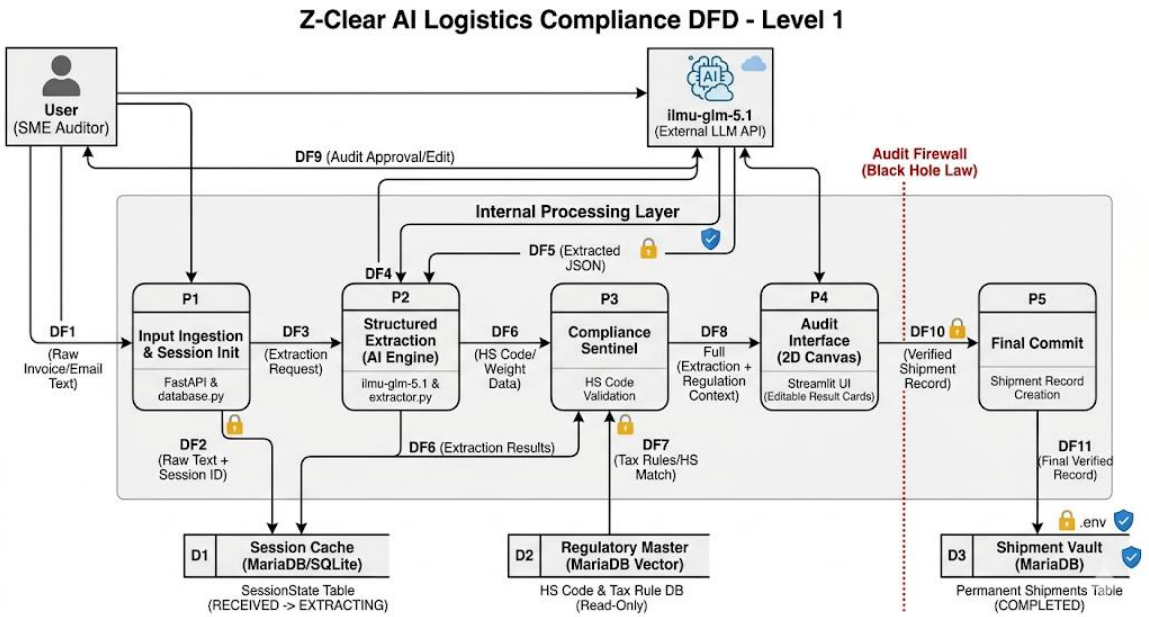
2.2.3. *Database:* MariaDB – Relational database for storing structured customs data, extracted fields (HS Code, weight, description), user records, and processing task history

2.2.4. *Cloud/Deployment:* Render – Cloud hosting platform for deploying backend services, frontend applications, and background workers, providing automated builds, scaling, environment management, and continuous deployment for the FastAPI microservices and web interfaces.

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

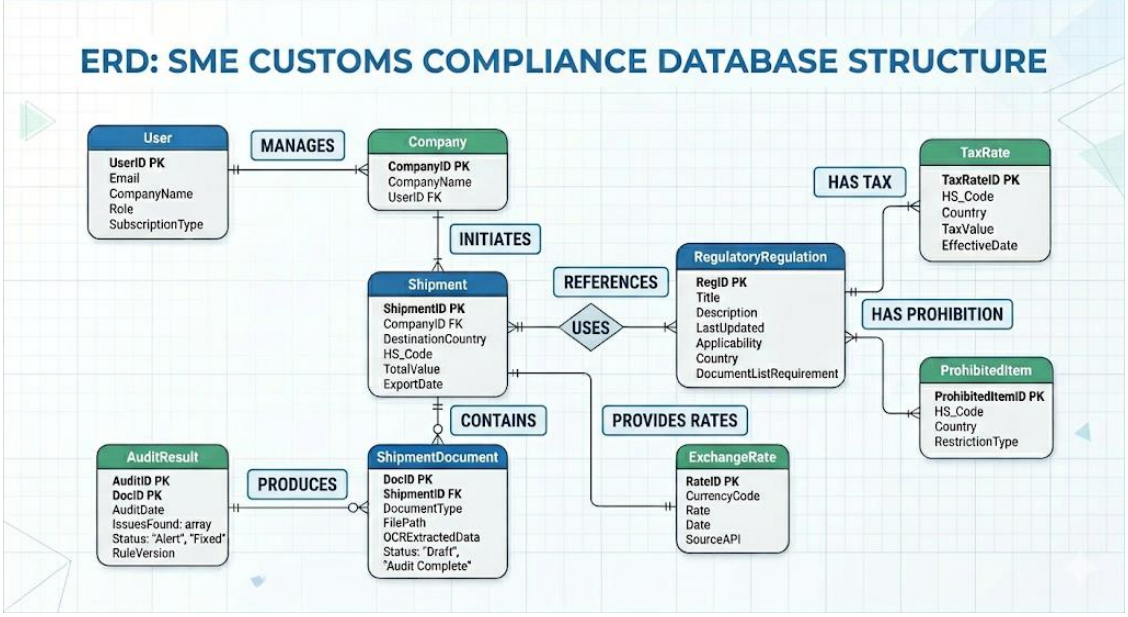
2.3.1.1. Data Flow Diagram (DFD)



2.3.2. **Normalized Database Schema**

© UMHackathon 2026

Email: umhackathon@um.edu.my



3. *Functional Requirements & Scope*

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. *Minimum Viable Product:*

3.1.1. Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026. ***[e.g. in Table]***

#	***Features***	***Description***
1	*Robust Handling of Unstructured Input:*	Automatically identifies and parses trade data from messy email bodies, informal chat logs, or shorthand notes, regardless of inconsistent formatting or non-standard layouts.
2	*One-Click Key Data Retrieval:*	Enables single-click extraction of essential customs clearance information, such as Goods Description, HS Code, and Weight, directly from the provided raw text.
3	*Shorthand & Jargon Recognition:*	Utilizes GLM to interpret industry-specific abbreviations and handwritten-style text inputs, normalizing them into standardized trade terminology suitable for official records.

© UMHackathon 2026

Email: umhackathon@um.edu.my

4	*Automatic Data Cleaning & Formatting:*	Transforms chaotic text descriptions into organized, structured fields (e.g., standardizing "Wt: 80kg" into a numerical weight value), ensuring the output is ready for direct database storage.

3.2. *Non-Functional Requirements (NFRs)*

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	***Requirements (e.g.)***	***Implementation (e.g)***
Scalability	System must handle increasing number of document submissions and concurrent users without performance degradation	Cloud-native deployment with horizontally scalable backend services (FastAPI containers), API Gateway load balancing, and stateless microservices design
Reliability	System must ensure continuous operation with minimal downtime and no data loss during processing failures	High-availability architecture with retry mechanisms, fallback parsing engine when GLM fails, and task state persistence in MariaDB
Maintainability	System should allow easy updates, debugging, and independent deployment of services	Modular microservice architecture with clear separation (document service, task service, data service), versioned APIs, and centralized logging/monitoring
Token Latency	AI response time must remain within acceptable limits for real-time user interaction (low processing delay per document)	Prompt optimization, input chunking before sending to GLM from Z.ai, caching repeated patterns, and async processing pipeline
Cost Efficiency	Minimize operational costs of AI inference and cloud infrastructure while maintaining performance	Token reduction via prompt compression, selective GLM invocation (only for complex inputs), use of batch processing,

© UMHackathon 2026

Email: umhackathon@um.edu.my

		and auto-scaling cloud resources based on demand

3.3. *Out of Scope / Future Enhancements*

Do identify the high-level features that the team planned for "Grand Vision" but not part of this UMHackathon 2026. [e.g. below]

a. Generative Document Drafting: While the current version identifies missing paperwork, future iterations will implement Generative AI to automatically draft high-accuracy templates. By integrating with the user's ERP or sales software, the AI will pre-fill complex forms (e.g., *Commercial Invoices* or *Certificates of Origin*) based on extracted transaction data, reducing the user's role to a final manual review.

b. Predictive Delay & Risk Analytics: The system will evolve to incorporate Predictive Analytics by analyzing historical port data, weather patterns, and real-time congestion reports. This will allow the AI to warn SMEs of potential shipment delays before they occur and suggest alternative routes or ports to avoid "demurrage" (storage) fees.

c. Blockchain-Based Traceability: To further enhance compliance, the system could integrate with Blockchain technology. This would create an immutable digital trail for every document and shipment, providing customs authorities with verified "Proof of Origin" and reducing the likelihood of physical inspections or audits.

d. Regulatory Monitoring and Law Capture: Continuously monitor global trade portals and government gazettes to capture latest laws and tax regulations. Using AI summarizations, converts complex legal updates into clear, real time alerts, ensuring SMEs never miss a change in export requirements

4. *Monitor, Evaluation, Assumptions & Dependencies*

4.1. *Technical Evaluation:*

4.1.1. **Grayscale Rollout:** New features—such as the automated regulatory alert system—are released to 5% of real users initially. System stability and feedback are closely monitored, with the rollout gradually increased to 20%, 50%, and eventually 100% if no issues are detected.

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.1.2. **Monitoring & Validation:** During the rollout, the team tracks the AI's diagnostic accuracy and latency. If technical anomalies or "False Negatives" in document auditing occur, the rollout is paused to ensure the system remains reliable for high-stakes customs compliance.

4.1.3. **A/B Testing:** After a feature passes the grayscale phase, A/B testing is conducted to determine which variants of two strategies perform better. For example, comparing two different NLP extraction models to see which identifies missing documentation more precisely in a real-world environment.

4.1.4. **Strategy Optimization:** Based on the results of the A/B test, the higher-performing variant is deployed as the standard, ensuring the AI continuously evolves to meet the specific documentation needs of SMEs.

4.1.5. Priority Matrix:

Priority	Trigger Conditions	Actions
P1 – Critical	GLM service failure (Z.ai), MariaDB corruption, system-wide outage, error rate > 5%	Activate fallback parser, stop/limit traffic, rollback to stable release, immediate alert
P2 – High	High latency (>10s GLM response), backlog > 500 tasks, partial pipeline failure	Auto-scale services, optimize chunking, switch to batch mode, send alerts
P3 – Medium	Slow DB queries, increased errors in specific document types, UI delay	Query optimization, caching, performance tuning
P4 – Low	Minor UI bugs, logging issues, non-critical defects	Logged for next release fix

4.2. Monitoring:

- **Application Monitoring:** Tracks API latency, error rate, and workflow status (Received → Completed). Alerts if error rate > 5% or latency > 3s.
- **AI Monitoring:** Monitors GLM response time, token usage, and failures. Timeout triggers fallback parsing.
- **Database Monitoring (MariaDB):** Tracks query performance, CPU usage, and storage health. Alerts on bottlenecks.
- **Infrastructure Monitoring (Render):** Monitors uptime, CPU, memory, and auto-restarts failed services.
- **Alert Levels:**
  P4 logged → P3 warning → P2 alert → P1 emergency rollback

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.3. *Assumptions:* *State key operational & environmental conditions that assumed to be valid while development of system and deployment.*

a. **Text-Based Input Requirement:** It is assumed that SME users will provide shipping data in a **pure text format** (e.g., copied-and-pasted text, CSV, or manual JSON input). The system assumes the user has already converted any physical or digital documents into a readable text string before inputting it into the AI prompt.

b. **Data Structure Integrity:** Since the system relies on text rather than visual layout, it is assumed that the text provided follows a logical sequence (e.g., clear headers and values) so that the **NLP (Natural Language Processing)** engine can accurately identify entities like "Invoice Number," "HS Code," and "Total Value" without the help of visual cues.

c. **Manual Entry Accuracy:** It is assumed that the user is responsible for the accuracy of the text provided. Unlike OCR-based systems that "read" what is there, a text-only system is highly sensitive to typos or copy-paste errors made by the user during the data entry process.

d. **Language Standard:** It is assumed that the input text primarily uses standard terminology in **English, Chinese, or Malay**, as the system may struggle to interpret non-standard abbreviations or heavily corrupted text strings that lack proper grammar or formatting.

e. **API and Web Access:** The system assumes consistent and open access to official government customs portals, gazettes, and international trade databases for the Regulatory Monitoring module to scrape and retrieve real-time updates.

f. **Internet Connectivity:** As a cloud-based AI system, it is assumed that the SME has a stable internet connection to sync with the central regulatory database and perform document audits in real-time.

4.4. *External Dependencies:*

Tools	*Purpose*	*Risks*
Z Ai GLM API	*It should be the core LLM reasoning engine - that will handle the prompt inference, multi-step reasoning and generating structured output*	*High- in case of GLM API rate limitation - core product will fail. So, implement a mitigation strategy [e.g. graceful degradation state from users]*

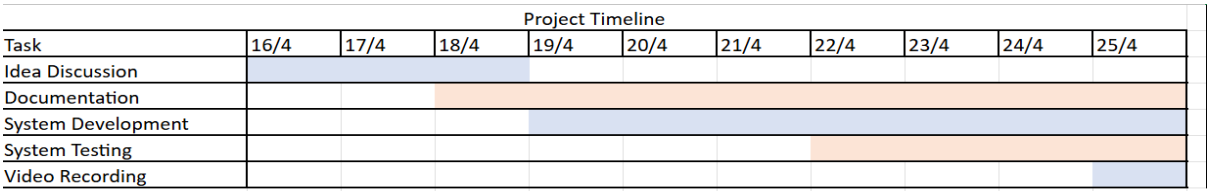
© UMHackathon 2026

Email: umhackathon@um.edu.my

5. *Project Management & Team Contributions*

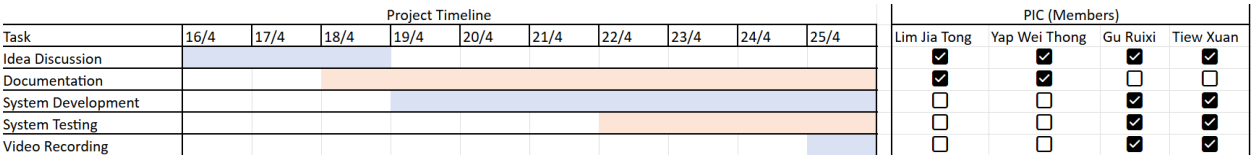
The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. *Project Timeline:*



5.2. *Team Members Role:*

Members	*Roles*
Lim Jia Tong	*Documentation*
Yap Wei Thong	*Documentation*
Tiew Xuan	*System development and deployment*
Gu Ruixi	*System development and deployment*



© UMHackathon 2026

Email: umhackathon@um.edu.my